

Rapport de stage : "There is no artificial intelligence yet"

Encadré par : Marc CHAUMONT et Gérard SUBSOL

Aymane JEBARI

August 25, 2023

Table des matières

1	Introduction :	2
2	Définitions de l'intelligence :	2
2.1	François Chollet - "On the measure of Intelligence" :	2
2.2	Pei Wang - "On Defining Artificial Intelligence" :	3
3	ARC Benchmark :	4
3.1	Description de la base de données :	4
3.2	Connaissances de base pour le challenge ARC :	4
3.3	ARC et Intelligence :	5
3.4	Autres variantes de ARC :	5
3.4.1	ConceptARC Benchmark :	5
3.4.2	MiniARC :	5
4	Approches étudiées :	6
4.1	Approche DSL (Domain specific language DSL approach) :	6
4.2	Approche de Minimum description length MDL :	6
4.3	Approche de DreamCoder :	7
5	Comparaison des approches et recul vis à vis la définition de l'intelligence :	8
6	Travaux réalisés :	8
6.1	Améliorations apportées au modèle DreamCoder :	8
6.1.1	Réseau de neurones :	8
6.1.2	Augmentation des données :	9
6.1.3	Métriques considérées pour l'évaluation du réseau de neurones :	9
6.2	Résultats obtenus :	10
6.2.1	Comparaison entre le réseau de neurones original et la nouvelle architecture :	10
6.2.2	Statistiques sur l'augmentation des données :	10
6.2.3	Paramètres d'entraînement et résultats:	10
6.3	Pistes d'amélioration :	11
6.3.1	Apprentissage auto-supervisé :	11
6.3.2	Phase d'abstraction :	11
6.3.3	Énumération des programmes :	11
7	Recul vis à vis le raisonnement humain :	11
8	Conclusion :	12
	Références :	13

1 Introduction :

L'Intelligence Artificielle, communément désignée par le terme AI, est devenue une notion souvent utilisée de nos jours. Cependant, pouvons-nous véritablement définir précisément cette notion et est-il approprié de qualifier les modèles actuellement développés en tant que systèmes "intelligents"? Est-ce que la simple possession d'une mémoire étendue et de capacités d'extrapolation, telles qu'observées dans plusieurs modèles de Deep Learning contemporains, suffit à les qualifier d'intelligents? Par ailleurs, sommes-nous sur le point de créer des Intelligences Artificielles Générales (Artificial General Intelligence, [AGI](#)) ?

Tout au long de mon stage, j'ai tenté de répondre à ces interrogations en prenant comme point de référence le challenge [ARCathon](#). Bien que ce défi puisse sembler élémentaire pour les êtres humains, il demeure extrêmement complexe à résoudre pour les approches exclusivement basées sur l'IA. En effet, seules les méthodes quasi-brute force parviennent à obtenir des scores respectables, mais elles restent bien loin des performances humaines.

2 Définitions de l'intelligence :

2.1 François Chollet - "On the measure of Intelligence" :

François Chollet débute son article [1] en mettant en évidence le fait qu'il n'existe pas véritablement de définition claire et précise de l'intelligence. Les définitions existantes sont souvent implicites et teintées de subjectivité (prenons par exemple le [test de Turing](#) [2], que ChatGPT peut potentiellement réussir, mais est-ce vraiment une marque d'intelligence pour un modèle ?). Si nous aspirons à progresser dans la mesure de l'intelligence, il est essentiel de mettre en lumière les préjugés inhérents à ces définitions, de définir un objectif explicite permettant de quantifier l'intelligence de manière déterministe, voire de concevoir une formule mathématique à cette fin.

Selon Chollet, l'intelligence se manifeste à travers la capacité d'un agent à accomplir des objectifs dans divers environnements. Cette capacité repose fondamentalement sur la capacité à acquérir de nouvelles compétences, à résoudre des tâches qui n'ont pas été préalablement rencontrées lors de l'entraînement, et à généraliser ces compétences. Chollet pousse même son analyse jusqu'à élaborer une formule mathématique pour définir l'intelligence (figure 1).

$$I_{IS,scope}^{opt} = Avg_{T \in scope} \left[\omega_{T,\theta} \cdot \theta \sum_{C \in Cur_T^{opt}} \left[P_C \cdot \frac{GD_{IS,T,C}^{\theta}}{P_{IS,T}^{\theta} + E_{IS,T,C}^{\theta}} \right] \right]$$

Figure 1: Formule d'intelligence proposée par François Chollet

Dans cette formule :

- $P_{IS,T}^{\theta}$ (**Priors**) représente la quantité d'informations pertinentes contenues initialement dans le système (connaissances de base pertinentes).
- $E_{IS,T,C}^{\theta}$ (**Experience**) représente l'expérience qu'a gagnée le système en s'exposant aux données d'entraînement. Elle mesure la quantité d'informations pertinentes reçues par le système au sujet de la tâche au cours de son entraînement.
- $GD_{IS,T,C}^{\theta}$ (**Generalization difficulty**) réfère au défi d'avoir une bonne performance sur des tâches dans de nouvelles situations au-delà des tâches d'entraînement. Si le comportement requis pour résoudre les tâches d'évaluation est significativement différent du comportement optimal le plus simple pour résoudre les tâches d'entraînement, la difficulté de généralisation est élevée.

- $\sum_{C \in \text{Cur}_T^{\text{opt}}} \left[P_C \cdot \frac{GD_{IS,T,C}^\theta}{P_{IS,T}^\theta + E_{IS,T,C}^\theta} \right]$. Pour une tâche fixe, cette somme est réalisée sur toutes les séquences d'interactions (**curriculum**) que le système a eu avec cette tâche et elle est pondérée par la probabilité de chaque interaction.
- $\omega_{T,\theta}$ représente une valeur subjective accordée à la résolution de la tâche T avec un seuil de précision θ
- $\omega_{T,\theta} \cdot \sum_{C \in \text{Cur}_T^{\text{opt}}} \left[P_C \cdot \frac{GD_{IS,T,C}^\theta}{P_{IS,T}^\theta + E_{IS,T,C}^\theta} \right]$ représente finalement la contribution de chaque tâche à l'intelligence du système.

En des mots simples, cette formule sous-entend que l'intelligence se mesure par la rapidité avec laquelle un apprenant transforme son expérience et ses connaissances de base en de nouvelles compétences pour d'autres tâches qui exigent adaptation et tolérance à l'incertitude. De plus, l'intelligence d'un système n'augmente pas uniquement lorsque la tâche à accomplir est plus complexe à résoudre, mais également lorsque les connaissances de base avant l'entraînement et l'expérience accumulée pendant l'entraînement sont limitées.

D'ailleurs, cette définition s'applique tout aussi bien à nous, les humains. Nous sommes davantage impressionnés par quelqu'un qui parvient à résoudre une tâche que nous considérons comme difficile, surtout si cette personne est jeune (probablement avec peu d'expérience) et provient d'un domaine d'études distinct de celui requis pour résoudre la tâche (ce qui implique moins de connaissances de base liés à la tâche et une grande faculté d'adaptation).

2.2 Pei Wang - "On Defining Artificial Intelligence" :

Dans son papier [3] intitulé "On Defining Artificial Intelligence" publié en 2019, Pei Wang indique lui aussi qu'il n'y a pas une définition claire et nette de l'intelligence artificielle. Selon lui, si jamais une définition est proposée, il faut que celle ci répond à 4 critères essentiels :

1. Cohérence avec la définition qui existe dans le dictionnaire et indépendance vis à vis les propriétés spécifiques à l'être humain qui peuvent être biologiques, historiques, sociales, etc.
2. L' "intelligence" ne doit pas être définie en termes d'autres concepts vagues, tels que "esprit", "pensée", "cognition", "sagesse", "conscience", etc. Mais bien sûr certains concepts utilisés dans la définition peuvent ne pas avoir de définition exacte eux-mêmes, et exiger leur définition entraînerait une récursion à l'infini. Nous devons nous arrêter quelque part et dépendre d'une compréhension commune de certains concepts comme point de départ d'un processus de définition.
3. Cette définition devrait fixer un objectif clair pour la recherche et fournir des orientations pour les travaux à venir.
4. La définition doit être la plus simple possible puisque les hypothèses simples sont supposés avoir une grande probabilité d'être correctes (**Occam's razor principle**)

Mais Il arrive souvent qu'une définition soit meilleure selon une norme mais moins bonne selon une autre.

La définition proposée par Pei Wang est la suivante : **L'intelligence est la capacité d'un système de traitement de l'information à s'adapter à son environnement tout en fonctionnant avec des connaissances et des ressources insuffisantes.**

Cette définition porte plusieurs implications sur un système dit "intelligent" :

- En premier lieu, il est essentiel que le système ait une taille finie et une capacité de traitement relativement constante dans le temps. Cela permet d'éliminer les modèles purement théoriques. De plus, il est impératif que le système soit "ouvert" et en mesure d'accepter de nouvelles tâches, à condition qu'elles soient conformes au format d'entrée requis. En outre, le système doit être opérationnel à la demande, ce qui signifie qu'il ne doit pas être fonctionnel uniquement lorsqu'il est préconfiguré pour des tâches spécifiques.

- Deuxièmement, l’environnement dans lequel le système opère peut être en constante évolution en fonction des tâches qui lui sont attribuées. Cette évolution dynamique signifie que le système peut être mis au défi par de nouvelles expériences et exigences.
- Enfin, il est important de noter que l’apprentissage du système n’a pas besoin d’être parfait. Cependant, le système doit démontrer une réelle capacité d’adaptation. Dans ce contexte, l’accent n’est pas seulement mis sur les résultats optimaux, mais également sur l’effort déployé pour résoudre une tâche donnée. Par exemple, si les nouvelles tâches ne sont pas directement liées aux expériences passées du système, il n’est pas possible d’évaluer sa performance en termes de résultats, mais plutôt en termes de mesures prises pour s’adapter à la situation.

3 ARC Benchmark :

3.1 Description de la base de données :

Afin d’établir une base de données permettant d’évaluer l’intelligence d’un système, Chollet propose dans son article [1] le corpus ARC (Abstraction and Reasoning Corpus). Ce jeu de données comprend 400 tâches d’entraînement et 400 tâches d’évaluation, ainsi que 100 tâches tenus secrets, réservés à l’évaluation finale du système. Cela garantit que le développeur du modèle ne dispose pas, en plus des connaissances de base nécessaires à la résolution de la tâche, d’informations préalables sur les exemples sur lesquels il sera évalué. En outre, chaque tâche est censée être unique, bien qu’elle soit fondamentalement basée sur des connaissances de base. Et chaque tâche comprend 2 à 5 exemples d’entraînement, chacun composé d’une grille d’entrée et d’une grille de sortie. L’objectif est de comprendre comment obtenir la grille de sortie à partir de la grille d’entrée et d’appliquer la règle trouvée à un ou plusieurs exemples d’évaluation. Quelques exemples des tâches du dataset d’entraînement sont représentés dans la figure 2 et 3

3.2 Connaissances de base pour le challenge ARC :

Pour aborder avec succès les tâches complexes d’ARC, Chollet indique que la maîtrise de quatre catégories de connaissances de base est suffisante :

1. La première catégorie concerne les objets (*Objectness priors*) :
 - **Cohésion des objets** (*Object cohesion*) : Capacité à regrouper des éléments dans des grilles en se basant sur des règles de continuité comme la cohérence des couleurs ou la proximité spatiale. Cela suppose la capacité à diviser les grilles en zones distinctes ou en partitions.
 - **Persistance des objets** (*Object persistence*) : L’idée que les objets restent identifiables malgré les perturbations telles que le bruit ou l’occultation par d’autres objets. Souvent, les objets d’entrée persistent dans la grille de sortie, parfois sous des formes modifiées.
 - **Contact entre objets** (*Object influence via contact*) : De nombreuses tâches impliquent des interactions physiques entre les objets, comme déplacer un objet jusqu’à ce qu’il touche un autre, ou étendre une ligne jusqu’à ce qu’elle entre en contact avec un autre objet.
2. La deuxième catégorie concerne les nombres et le comptage (*Numbers and Counting priors*) :
 - Les objets peuvent être reconnus et séparés, par exemple par leur forme, leur apparence, leur mouvement ...
 - Les objets peuvent être comptés et triés par nombre, taille, forme ...
 - Les mathématiques de base comme l’addition, la soustraction et la comparaison de petits nombres d’objets sont possibles.
3. La troisième catégorie concerne l’orientation vers un objectif (*Goal-directness prior*) :

- Bien que ARC n'intègre pas le concept de temps, bon nombre des grilles d'entrée/sortie peuvent être efficacement modélisées par les humains comme les états de départ et d'arrivée d'un processus impliquant une intentionnalité. Ainsi, le principe d'orientation vers l'objectif pourrait ne pas être strictement nécessaire pour résoudre l'ARC, mais il est probablement utile.

4. La quatrième catégorie concerne la géométrie de base (*Basic geometry and Topology priors*) :

- Lignes et formes géométriques simples comme les rectangles, les triangles et les cercles
- Les formes géométriques d'un même type peuvent être distinguées en tant que telles, même après d'éventuelles transformations.
- Les objets peuvent être symétrisés, tournés, translatés, déformés...
- Les objets peuvent être assemblés, démontés et les pièces qui constituent des objets plus grands peuvent être détectées.
- Les objets peuvent être répétés
- Les points peuvent être reliés, par exemple, à un chemin.
- Les différences de distance peuvent être détectées

3.3 ARC et Intelligence :

Le challenge ARC peut être considéré comme un test psychométrique d'intelligence. Son utilité s'étend à la fois pour les êtres humains et les systèmes d'IA visant à reproduire une forme d'intelligence générale similaire à celle humaine. En termes de structure, ARC emprunte le format des [matrices progressives de Raven](#), un modèle classique de test de quotient intellectuel (IQ) datant des années 1930. ARC est accessible tant aux individus qu'aux machines, en veillant à ce que la résolution des problèmes ne nécessite que peu de connaissances de base. Un grand avantage d'ARC est son objectif de mesurer la capacité de généralisation chez les concepteurs des systèmes AI, plutôt que de se focaliser sur des compétences spécifiques à des tâches particulières. Pour ce faire, il est basé sur des tâches uniques dans son ensemble des tâches secrètes, supposées être inconnues du développeur lors de la conception du système. Cette approche permet d'évaluer une forme de généralisation qualitative et étendue, à travers des tâches abstraites exigeant une compréhension minimale de la part du développeur. ARC se distingue également par sa capacité à définir explicitement les connaissances de base requises pour sa résolution, connaissances maîtrisées par les êtres humains. Cela facilite une comparaison équitable de l'intelligence générale entre les êtres humains et les systèmes d'IA. Ainsi, il favorise une évaluation objective et équilibrée de l'intelligence.

3.4 Autres variantes de ARC :

3.4.1 ConceptARC Benchmark :

Le jeu de données ConceptARC [4] adopte la même philosophie que les tâches ARC, mais il est structuré en regroupant les tâches en classes. Chaque classe est centrée autour d'un concept clé qui sert de base pour résoudre les tâches. Chaque classe contient 16 tâches distinctes. La liste des concepts utilisés ainsi que les performances de certaines méthodes sur ce jeu de données sont présentées dans la figure 4. ConceptARC semble relativement être plus facile qu'ARC puisqu'il y a souvent qu'un seul concept qui est utilisé. Des exemples de ConceptARC sont données dans la figure 5. L'article [4] fournit plus de détails sur le dataset et les résultats correspondantes.

3.4.2 MiniARC :

Le jeu de données en question [5] se compose de 150 tâches distinctes. Toutefois, il convient de noter que la dimension des grilles d'entrée et de sortie de chaque exemple est préalablement fixée à 5x5. Cette simplification a pour effet de réduire la complexité du problème : la dimension est restreinte, la taille de la grille de sortie est prédéterminée et la tâche du modèle consiste essentiellement à prédire la couleur de chaque pixel.

De plus, ces tâches sont classées en 6 catégories au sein de MiniARC, et sont : "movement", "color", "object", "number", "geometry", et "common-sense". Des exemples représentatifs de chacune de ces

catégories sont illustrés dans la figure 6. Les détails concernant ce jeu de données ainsi que les résultats correspondants sont exposés de manière approfondie dans l'article référencé [5].

4 Approches étudiées :

Lors de la première [compétition d'ARC sur kaggle](#) en 2020, l'approche la plus réussie a atteint 20,6 %. Ce record a depuis été dépassé à trois reprises avec des scores de 28,5 %, 30,4 % et 31,4 %. Néanmoins, les méthodes qui ont permis d'obtenir ces résultats n'étaient pas excessivement innovantes, mais plutôt des combinaisons simples des meilleurs codes disponibles publiquement. Il est donc possible d'affirmer qu'il y a eu peu de progrès depuis le premier défi il y a plus de trois ans, du moins en ce qui concerne le meilleur score sur cet ensemble de tests cachés. Voici un descriptif des principales méthodes que j'ai pu investiguer :

4.1 Approche DSL (Domain specific language DSL approach) :

Il s'agit de l'approche du vainqueur dans la compétition ARC sur kaggle (2020) [6]. La solution qu'il présente repose principalement sur un langage de domaine spécifique (DSL). Ce DSL applique un maximum de quatre transformations unaires parmi un total de 142. Ces transformations sont basées sur 42 fonctions distinctes, certaines de ces fonctions ayant plusieurs variantes. Pour garantir une efficacité optimale, il énumère ces transformations de manière à éviter les répétitions, puis les combine en les empilant avec soin, en accordant la priorité aux échantillons d'entraînement. Toute cette opération est réalisée de manière performante en utilisant le langage de programmation C++, sans nécessiter de dépendances extérieures. De plus, le processus s'exécute en parallèle, ce qui contribue à accélérer le traitement. Un planificateur simple vise ensuite à exploiter au mieux l'allocation de ressources limitée, consistant en un budget de 9 heures de calcul et 16 Go de mémoire. Parmi les transformations définies :

- Cut (image) → list of images : déduit la couleur de fond de l'image et renvoie les parties connexes de celle-ci (objets)
- filterCol (image, color) → image : supprime toutes les couleurs à l'exception de celle donnée comme paramètre
- colShape (image, color) → image : Change la couleur des pixels qui ne sont pas de couleur noire en "color"

Cette approche a permis à Johannes Sokrates Wind "icecuber" d'obtenir le meilleur score 20.8% sur le dataset gardé secret de la compétition. Pourtant, l'approche elle-même est basée sur une recherche brute force, et Johannes lui-même affirme que l'objectif de la compétition est loin d'être atteint. "Unfortunately, I don't feel like my solution itself brings us closer to AGI."

4.2 Approche de Minimum description length MDL :

C'est une approche développée par Sébastien Ferré (rechercheur à IRISA). L'approche en soi n'est pas évidente à comprendre, il faut reprendre le papier [7] pour bien l'assimiler. Mais voici un récapitulatif de l'approche en l'appliquant à la tâche "b94a9452.json" de la base d'entraînement (figure 7):

1. On commence par vectoriser nos grilles input/output en fonction des expressions définies dans la figure 8.

L'objectif poursuivi consiste à développer un modèle de type "Pair" ayant la capacité de décrire l'ensemble des exemples, comprenant à la fois les grilles d'entrée et de sortie, pour chaque tâche donnée. Il est important de noter que ce modèle doit présenter la particularité que les expressions utilisées pour décrire les sorties dépendent des entrées respectives. En d'autres termes, le modèle qui caractérise la relation entre les entrées et les sorties doit incorporer des expressions qui sont adaptées à chaque exemple spécifique.

Concrètement, l’objectif final est d’aboutir à un modèle tel que présenté dans la figure 9. Dans ce schéma, les ”?” symbolisent des variables qui prennent leur valeur en fonction de chaque exemple individuel, permettant ainsi au modèle de s’ajuster en conséquence.

En des mots simples, ce modèle dit : ”trouver deux rectangles pleins empilés sur un fond noir dans la grille d’entrée, puis générer une grille de sortie dont la taille est la même que la celle du second objet (défini dans la grille input) et dont la couleur de fond est celle du premier objet, et enfin rajouter à cette grille un rectangle plein dont la taille est la même que celle du premier objet, dont la couleur est la même que celle du second objet et dont la position est la différence des tailles des deux objets”. Ce qui revient à rogner la grille sur le rectangle coloré le plus grand. Ensuite, on inverse les couleurs des deux rectangles.

2. Ensuite, l’approche repose sur le principe fondamental du [Minimum Description Length](#) (longueur minimale de description) et attribue à chaque type élémentaire une valeur représentant sa longueur de description, conformément aux règles énoncées dans la figure 10. Un point important à souligner est que plus la définition d’un objet est concise, plus sa longueur de description est réduite. Par exemple, il est plus judicieux de définir un rectangle présent dans une grille en utilisant le type **Rectangle** plutôt qu’une suite de pixels, chacun étant du type **Point**. De plus, la longueur de description d’un modèle est la somme des longueurs des éléments qui le composent.
3. Initialement, le modèle décrivant chaque grille est le plus simple possible, se présentant comme une liste de pixels de type **Point**. Ensuite, le modèle est progressivement affiné à travers une série d’itérations visant à aboutir à une description la plus concise possible pour toutes les grilles. Il est important de noter que lors de chaque itération, l’ensemble des grilles est pris en compte simultanément, et le modèle affiné ainsi élaboré est appliqué de manière uniforme à toutes les grilles.
4. Le processus débute par l’introduction d’objets de type rectangle accompagnés d’un masque cohérent dans le modèle des grilles d’entrée et de sortie lorsque cela est faisable. De plus, les expressions présentes dans la grille de sortie sont substituées par des expressions qui dépendent de la grille d’entrée lorsque cela est possible, contribuant ainsi à la réduction de la longueur de description du modèle. Les itérations spécifiquement sélectionnées par le modèle pour résoudre la tâche b94a9452.json sont clairement présentées dans la figure 11. Cette illustration démontre la transition d’une longueur de description normalisée de 2 au début du processus à une longueur de 0.199 à l’itération 20.

Cette approche a pu résoudre 41/400 tâches de la base de données d’entraînement et 10/400 tâches de la base d’évaluation. Mais elle n’a pu résoudre que 2% des tâches gardées secrètes.

4.3 Approche de DreamCoder :

C’est l’approche [8] à laquelle j’ai consacré le plus de temps. Le schéma de l’architecture est représenté dans la figure 12. Voici un résumé de l’approche

On commence tout d’abord par définir une librairie L qui contient des connaissances de base et des primitives (opérateurs, fonctions élémentaires ...) qui seront à la base de l’apprentissage du modèle, ensuite le modèle est basé sur trois cycles:

- Cycle «Wake» ou phase d’énumération : les programmes qui sont les plus probables à résoudre les tâches d’entraînement sont testés. Un réseau de neurones Q (entraîné au niveau du 3ème cycle) permet de choisir les programmes les plus probables. Ainsi, le réseau Q oriente l’exploration des solutions.
- Cycle «Sleep: Abstraction» ou phase d’abstraction : les programmes trouvés dans le cycle précédent sont factorisés pour essayer d’extraire les sous-programmes qui reviennent le plus dans ces programmes. Ensuite, ces sous-programmes sont ajoutés à la librairie L . Comme ça les programmes trouvées seront plus simples à écrire, et surtout à retrouver, en fonction des nouvelles composantes récemment ajoutés à la librairie.
- Cycle «Sleep: Dreaming» ou phase d’entraînement du réseau de neurones : Dans ce cycle, le réseau de neurones Q est entraîné sur les composantes de la librairie L . Comme ça, on aura pour

chaque tâche une distribution sur l'espace des programmes. Cette approche facilite la recherche de programmes en évitant l'émergence d'une distribution d'équiprobabilité dans le choix des différentes composantes de la librairie L , comme c'est le cas dans une recherche brute force.

Le grand avantage de cette approche, c'est qu'on est face à un modèle qui apprend au fur et à mesure qu'il résout des tâches. Il ne s'agit pas vraiment d'un modèle qui est défini de façon quasi-déterministe à l'avance comme c'est le cas pour les deux approches précédentes, mais c'est un modèle qui s'adapte à la tâche rencontrée.

5 Comparaison des approches et recul vis à vis la définition de l'intelligence :

La shéma de la figure 13 résume les avantages, les inconvénients et quelques points à améliorer pour chaque approche :

6 Travaux réalisés :

6.1 Améliorations apportées au modèle DreamCoder :

6.1.1 Réseau de neurones :

Le réseau de neurones original de DreamCoder est illustré dans la figure 12. Étant donné que les dimensions des grilles varient, une première étape consiste à appliquer un remplissage (padding) aux grilles d'entrée et de sortie de chaque exemple. Cette opération vise à les convertir en grilles de dimensions fixes, soit $11 \times 30 \times 30$ (la première dimension pour les 10 couleurs et le padding). Subséquemment, pour chaque exemple, les grilles d'entrée et de sortie sont concaténées le long d'une nouvelle dimension, aboutissant à un tenseur de taille $2 \times 11 \times 30 \times 30$.

Par la suite, un encodeur est intégré, composé de multiples couches de convolution et de couches de MaxPooling, comme présenté dans la figure 15. Cette procédure est faite pour chaque exemple, et par la suite, une moyenne est calculée sur l'ensemble des exemples. En fin de compte, des couches entièrement connectées (Fully Connected, FC) sont ajoutées au réseau, ce qui produit un vecteur de taille correspondant au nombre de primitives dans la librairie L . Ce vecteur représente les probabilités associées à l'existence de chaque primitive dans le programme potentiellement capable de résoudre la tâche. De plus, en introduisant un paramètre dans la commande, il deviendra possible de transformer la sortie du modèle en une matrice contenant une distribution de probabilités conditionnelles. En d'autres termes, cela permettra d'obtenir la probabilité d'une primitive donnée en fonction de la primitive précédemment utilisée.

Dans la nouvelle architecture que j'ai proposée (figure 16), deux améliorations significatives ont été apportées :

- Au lieu d'appliquer un remplissage (padding) aux grilles, on introduit des couches de convolution qui s'ajustent à la taille des grilles (utilisant l'adaptive average pooling). Cette modification permet d'adapter le modèle aux différentes dimensions de grilles sans nécessiter de padding artificiel.
- L'encodage des grilles de chaque exemple (entrée et sortie) est désormais effectué de manière distincte. Cette approche simplifie la tâche de l'encodeur, qui est maintenant chargé d'extraire des informations spécifiques à chaque grille, au lieu de chercher directement à établir un lien entre les deux grilles (comme dans l'ancienne architecture). Il convient de noter que les poids des composantes de l'encodeur sont partagés entre toutes les grilles.

Ces ajustements visent à améliorer les performances et la capacité d'apprentissage du modèle en adaptant son fonctionnement aux caractéristiques des données d'entrée et en simplifiant la tâche de l'encodeur.

6.1.2 Augmentation des données :

Le plus grand challenge dans le jeu de données d'ARC, c'est que le nombre d'exemples par tâche est faible (entre 2 à 5 exemples), ce qui rend la tâche d'entraînement du réseau de neurones très difficile. Voici quelques pistes d'augmentation des données qui ont été considérées :

- **Changement des couleurs** : Prenons l'exemple de la figure 17. Dans cette tâche, il fallait trouver le pixel de la couleur minoritaire et puis ajouter un contour rouge autour de ce pixel en supprimant les autres. On voit donc bien que la couleur rouge va très probablement exister dans le programme qui permet de résoudre la tâche. Ainsi si jamais on change la couleur rouge dans la grille de sortie le programme qui résout la tâche doit changer également. Voilà pourquoi il fallait trouver des règles pour le changement des couleurs tout en limitant le risque de la modification du programme. Les règles trouvées sont les suivantes :
 1. Si une couleur existe dans toutes les grilles d'entrée ou dans tous les grilles de sortie, il ne faut pas changer cette couleur.
 2. Si une couleur existe à la fois dans une grille d'entrée et de sortie, le changement de la couleur dans la grille d'entrée doit entraîner le changement de cette couleur dans la grille de sortie (figure 18).
 3. Si une couleur existe dans la grille de sortie mais pas dans la grille d'entrée, on ne change pas cette couleur.

Malgré le respect de ces règles, il subsiste toujours un risque de modification du programme. Mais, il semble que le nombre de tâches où cela est vrai soit faible. Dans la figure 19, chaque couleur est associée à une autre couleur et donc le changement de couleurs, sans résolution de la tâche, devient un peu plus compliqué.

- **Dilatation des grilles** : Une autre approche pour augmenter les données est d'agrandir les grilles d'entrée et de sortie en transformant chaque pixel de la grille en un rectangle de 2×2 pixels, tout en maintenant la couleur du pixel. En général, cela n'altère pas le programme qui résout la tâche. Cependant, il est important de veiller à ce que la longueur et la largeur de la grille ne dépassent pas 15 pixels, afin que la grille agrandie respecte toujours la contrainte de taille (inférieure à 30×30).
- **Rotation des grilles** : Une dernière méthode d'augmentation des données consiste à faire pivoter les grilles carrées d'entrée et de sortie de 90° , 180° et 270° . J'ai choisi de considérer uniquement les grilles carrées, car dans de nombreuses tâches où des opérateurs logiques sont utilisés, la grille est divisée selon une direction spécifique (horizontale ou verticale), comme illustré dans la figure 20.

Même si le risque de modification du programme en augmentant les données est limité, pour éliminer ce risque entièrement, les données augmentées ne sont considérées que dans la phase d'entraînement du réseau de neurones et ne sont pas utilisés dans la phase d'énumération.

6.1.3 Métriques considérées pour l'évaluation du réseau de neurones :

- **La Cross Entropy Loss** : elle sert de fonction de perte durant l'entraînement du réseau, et peut également être employée à la fin pour évaluer la performance des prédictions du réseau sur les tâches spécifiques utilisées pour son entraînement.
- **Coefficient de corrélation de Spearman** : Ce coefficient évalue dans quelle mesure la relation entre deux variables peut être décrite par une fonction monotone, contrairement au coefficient de Pearson où cette fonction monotone doit obligatoirement être affine. Ce coefficient se révèle particulièrement pertinent pour calculer la corrélation entre le classement des primitives prédit par le réseau de neurones et le classement réel des primitives. Ce coefficient prend une valeur comprise entre -1 et 1.

6.2 Résultats obtenus :

6.2.1 Comparaison entre le réseau de neurones original et la nouvelle architecture :

Afin de comparer les performances des deux réseaux de neurones, un ensemble de 41 tâches résolues a été utilisé comme base d'entraînement pour les deux réseaux. Après l'entraînement, les probabilités prédites ont été utilisées pour calculer à la fois la perte de cross-entropy et le coefficient de Spearman par rapport à ces 41 tâches. Le graphique de la figure 22 illustre les valeurs du coefficient de Spearman et de la perte d'entropie croisée pour les deux architectures sur les tâches d'entraînement du réseau. Il est important de noter que plus le coefficient de Spearman est élevé, meilleures sont les prédictions fournies par le réseau, tandis que l'inverse est vrai pour la perte d'entropie croisée. Il convient de noter que ces valeurs ont été calculées uniquement en se basant sur les 20 primitives les plus probables prédites par chaque réseau.

On remarque que les performances de la nouvelle architecture sont légèrement supérieures à celles de l'architecture originale. En effet, pour la nouvelle architecture, la moyenne du coefficient de Spearman est plus élevée (0,34 contre 0,28), tandis que la moyenne de la perte d'entropie croisée est plus faible (0,20 contre 0,22). Par conséquent, lors de la phase d'augmentation des données, la nouvelle architecture du réseau de neurones est utilisée.

6.2.2 Statistiques sur l'augmentation des données :

Le graphe de la figure 23 représente le nombre d'exemple d'entraînement pour les tâches augmentées et les tâches non augmentées. Le tableau ci-dessous contient des statistiques sur le nombre d'exemples d'entraînement après avoir réalisé l'augmentation des données :

	moyenne du nombre d'exemples d'entraînement par tâche	écart type du nombre d'exemples d'entraînement par tâche
Tâches non augmentées	3.25	0.97
Tâches augmentées	23.54	15.54

Il est également important de souligner qu'il y a 24 tâches parmi les 400 qui n'ont pas été modifiées. Ces tâches consistent en des grilles non carrées avec une longueur ou une largeur supérieure à 15, et où les contraintes liées aux couleurs empêchent toute modification.

6.2.3 Paramètres d'entraînement et résultats:

L'objectif de cette étude était d'analyser les avantages de l'augmentation de données sur les performances du réseau neuronal et du modèle. Pour ce faire, j'ai maintenu les mêmes paramètres afin de comparer les résultats du modèle entre un réseau neuronal sans augmentation de données et le même réseau avec augmentation de données. Les paramètres pris en compte sont récapitulés dans le tableau suivant :

Nombre de cycles	temps d'énumération	nombre d'époques d'entraînement du réseau de neurones	Nombre de CPUs
7	8h	1400	16

Le graphique représenté dans la Figure 24 illustre l'évolution du nombre des tâches résolues par le modèle avec augmentation des données par rapport au modèle sans augmentation des données. Il est évident que les performances des deux courbes commencent à se stabiliser autour du quatrième cycle. Bien que le modèle sans augmentation des données montre de bonnes performances au cours des trois premiers cycles, c'est finalement le modèle avec augmentation des données qui atteint le plus grand nombre de tâches résolues (58 contre 55).

Il convient de souligner qu'en ce contexte, le code a été exécuté sur un serveur accessible à d'autres utilisateurs, ce qui aurait pu influencer les résultats. Bien que le choix d'une durée d'énumération puisse être plus pratique en tant que paramètre, explorer des méthodes permettant de déterminer un nombre fixe de programmes testés plutôt que de se baser uniquement sur le temps d'entraînement pourrait fournir une comparaison plus équitable. Ces résultats ont été obtenus en modifiant une partie du code de [DreamCoder sur Github](#). Le code modifié peut être consulté en suivant ce [lien](#).

6.3 Pistes d'amélioration :

6.3.1 Apprentissage auto-supervisé :

Lorsqu'il s'agit d'améliorer l'apprentissage du réseau de neurones, l'approche envisageable est l'augmentation des données. Une autre méthode intéressante à considérer serait la mise en place d'un processus d'apprentissage auto-supervisé au niveau du réseau de neurones. Cela signifie que, en raison du manque de données étiquetées (les solutions aux tâches ne sont pas connues à l'avance), le réseau pourrait utiliser les grilles d'entrée existantes ou en créer de nouvelles. Ensuite, le modèle serait capable de générer des programmes en utilisant les primitives déjà définies. En appliquant ensuite ces programmes aux grilles d'entrée, les grilles de sortie pourraient être obtenues. Cependant, il faut être prudent lors de la génération des programmes, car certains pourraient sembler complexes mais finalement être équivalents à la fonction identité (comme la composition de deux réflexions horizontales). L'objectif serait donc d'entraîner le modèle sur ces données générées, en espérant que cela améliorerait statistiquement l'apprentissage du réseau de neurones. Il est à noter que ce processus est déjà mis en œuvre dans d'autres domaines tels que le "list processing" (figure 25), mais pas encore pour "ARC".

6.3.2 Phase d'abstraction :

Le schéma présenté dans la figure 26 reprend les concepts qui ont été acquis au cours de la phase d'abstraction du modèle. On peut observer que certains de ces concepts ont une portée générale et s'avèrent particulièrement intéressants, tels que le concept **f10**, qui permet de générer une superposition de la grille d'entrée avec sa réflexion horizontale. Ce concept se révèle particulièrement utile dans la résolution de diverses tâches, où il est nécessaire de compléter les grilles en tirant parti de symétries spécifiques (comme illustré dans la figure 27).

Cependant, d'autres concepts se montrent plus spécifiques. Par exemple, le concept **f0** peut être pertinent uniquement pour des grilles d'entrée ayant une largeur fixe de 3. Ce concept traduit la concaténation de la grille d'entrée avec sa réflexion, du côté droit. Dans l'idéal, il aurait été souhaitable que les paramètres des primitives soient dépendants des caractéristiques des grilles, ou qu'ils restent des variables inconnues à découvrir lors de la phase d'énumération. Cette approche aurait permis de rendre les concepts appris plus généralisables et, par conséquent, plus pertinents pour plusieurs tâches.

6.3.3 Enumération des programmes :

Pour certaines tâches, le programme trouvé lors de la phase d'énumération peut être notablement non optimal en termes de longueur et de simplicité. Par exemple, il est possible d'utiliser une primitive qui modifie une couleur qui n'est même pas présente dans les grilles d'entrée. Par conséquent, il pourrait s'avérer avantageux de mettre en place un processus visant à éliminer les segments du programme qui ne sont pas particulièrement utiles.

7 Recul vis à vis le raisonnement humain :

Dans de nombreuses situations, pour résoudre des tâches complexes, un être humain adopte une approche qui consiste à examiner et analyser les exemples liés à la tâche. Ensuite, il émet des hypothèses et les teste en les ordonnant de la plus probable à la moins probable, afin de déterminer celle qui s'applique à tous les exemples. Par exemple, face à une tâche impliquant des grilles de sortie dont les dimensions sont le double de celles des grilles d'entrée, l'observateur pourrait initialement explorer l'hypothèse d'une dilatation des grilles d'entrée.

Dans le cadre du modèle DreamCoder, une approche similaire est adoptée. Le réseau de neurones agit comme un guide pour orienter la recherche, en s'appuyant sur son expérience préalable avec des tâches déjà résolues. Il formule ensuite des hypothèses qui sont soumises à vérification au cours de la phase d'énumération. Cependant, il est important de noter que le réseau de neurones n'atteint pas la même performance que notre cerveau, principalement pour la raison suivante :

- Notre expérience passée nous a permis de développer une familiarité avec les primitives définies au départ et les connaissances de base, tandis que le réseau de neurones est entraîné exclusivement sur des tâches ARC qui sont complexes et ne se résolvent pas aisément pour le modèle. Ensuite, chaque programme dans le cadre des tâches ARC est une composition de plusieurs

primitives, ce qui rajoute de la complexité au processus d'apprentissage. Bien que les composantes de la librairie soient définies de manière déterministe, dans le contexte de la recherche, nous dépendons entièrement du réseau de neurones pour nous guider, et ce dernier n'interagit pas avec les définitions déterministes des fonctions primitives. Ainsi, une question pertinente à se poser serait de savoir si le réseau de neurones devrait être préalablement entraîné sur des tâches simples, impliquant des programmes très courts, afin d'assimiler de manière progressive les transformations de chaque primitive de manière individuelle. Une approche d'auto-supervision, comme détaillée précédemment, pourrait être utilisée à cette fin.

8 Conclusion :

En conclusion, l'exploration de l'intelligence artificielle à travers le prisme du benchmark ARC a mis en évidence à la fois les complexités inhérentes et les limites des systèmes d'IA actuels. L'exploration des différentes approches, allant des stratégies de langage spécifique au domaine (DSL) jusqu'aux méthodes fondées sur la minimisation de la longueur de description (MDL), en passant par DreamCoder, a permis de mettre en lumière les subtilités qui sous-tendent le raisonnement, l'abstraction et la résolution de problèmes d'ARC.

A la fin de ce stage, il est désormais clair que, malgré les progrès considérables, l'expression "intelligence artificielle" demeure partiellement inadéquate. Même si les systèmes d'IA peuvent exceller dans la reconnaissance de motifs et le traitement de données, les processus sous-jacents sont souvent handicapés par un manque de profondeur et de contextualisation, des attributs inhérents à la pensée humaine. Comme le soulignent avec pertinence François Chollet et Pei Wang, l'intelligence dépasse largement la simple performance sur des tâches spécifiques ; elle inclut une compréhension plus large, une adaptabilité accrue et la compétence de raisonnement dans des domaines variés. Par ailleurs, les résultats prometteurs issus de l'application de la data augmentation insufflent une lueur d'espoir quant à l'amélioration des performances du modèle DreamCoder.

Références :

- [1] François Chollet. On the measure of intelligence. 2019. [Available here](#).
- [2] Mengmi Zhang, Giorgia Dellaferrera, Ankur Sikarwa, Marcelo Armendariz, Noga Mudrik, Prachi Agrawal, Spandan Madan, Andrei Barbu, Haochen Yang, Tanishq Kumar, Meghna Sadwani, Michele Pizzochero Stella Dellaferrera, Hanspeter Pfister, and Gabriel Kreiman. Human or machine? turing tests for vision and language. 2022. [Available here](#).
- [3] Pei Wang. On defining artificial intelligence. *Journal of Artificial General Intelligence*, 2019. DOI : 10.2478/jagi-2019-0002.
- [4] Victor Vikram Odouard Arseny Moskvichev and Melanie Mitchell. The conceptarc benchmark: Evaluating understanding and generalization in the arc domain. 2023. [Available here](#).
- [5] Donghyun Ahn Subin Kim, Prin Phunyaphibarn and Sundong Kim. Playgrounds for abstraction and reasoning. *NeurIPS 2022 Workshop nCSI Program Chairs*, 2022. [Available here](#).
- [6] Johan Sokrates Wind. Arc-solution. 2020. [Available here](#).
- [7] Sébastien Ferré. First steps of an approach to the arc challenge based on descriptive grid models and the minimum description length principle. 2021. [Available here](#).
- [8] Kevin Ellis, , Catherine Wong, Maxwell Nye, Mathias Sablé-Meyer, Luc Cary, Lucas Morales, Luke Hewitt, Armando Solar-Lezama, and Joshua B. Tenenbaum¹. Dreamcoder: Growing generalizable, interpretable knowledge with wake-sleep bayesian program learning. *Royal Society*, 381, 2020. [Available here](#).

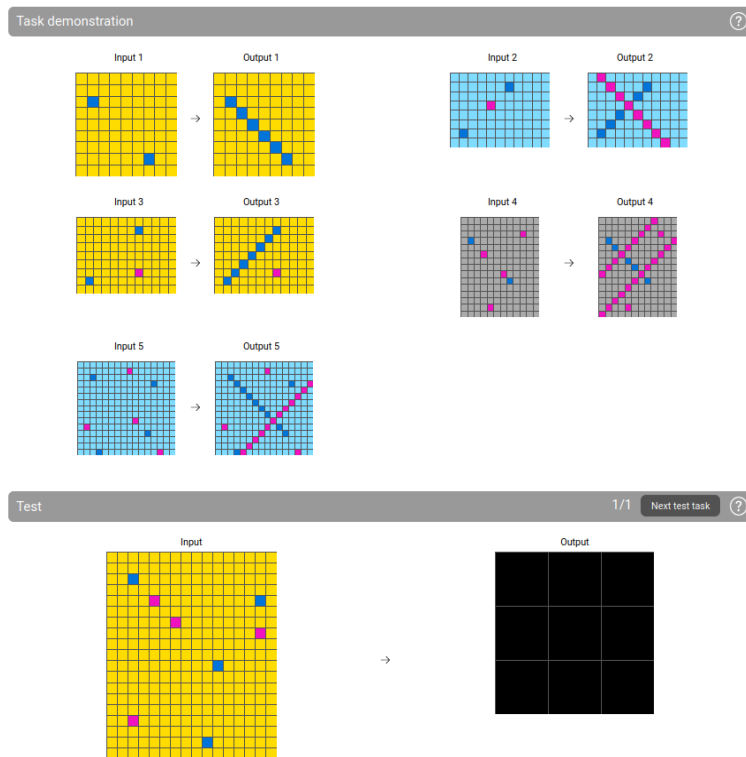


Figure 2: task 55783887.json du dataset d'entraînement d'ARC

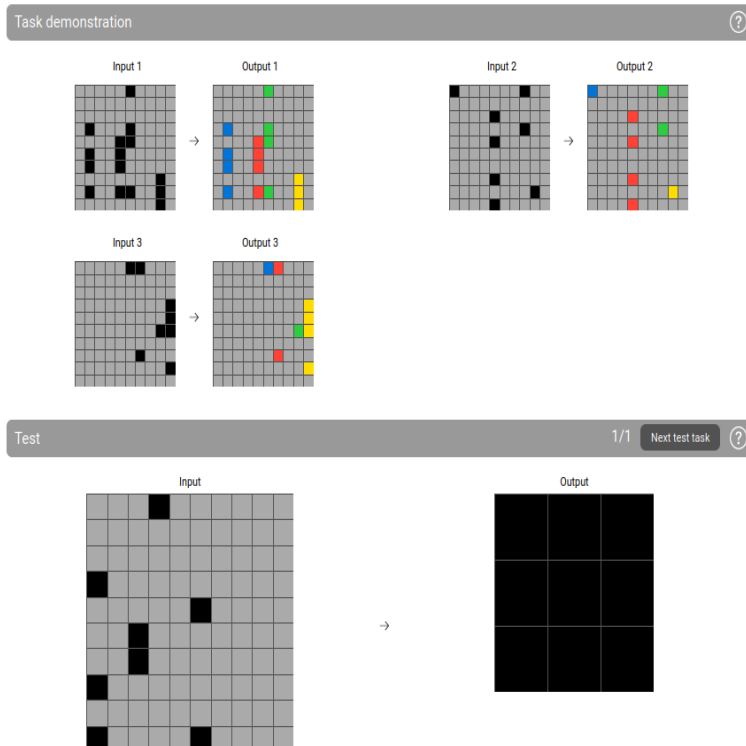


Figure 3: task 575b1a71.json du dataset d'entraînement d'ARC

Concept	Humans	ARC-Kaggle First Place	ARC-Kaggle Second Place	GPT-4
Above and Below	0.90	0.70	0.33	0.23
Center	0.94	0.50	0.20	0.33
Clean Up	0.97	0.50	0.20	0.20
Complete Shape	0.85	0.47	0.30	0.23
Copy	0.94	0.23	0.27	0.23
Count	0.88	0.60	0.40	0.13
Extend To Boundary	0.93	0.77	0.47	0.07
Extract Objects	0.86	0.43	0.43	0.03
Filled and Not Filled	0.96	0.73	0.43	0.17
Horizontal and Vertical	0.91	0.43	0.10	0.27
Inside and Outside	0.91	0.57	0.10	0.10
Move To Boundary	0.91	0.37	0.30	0.20
Order	0.83	0.27	0.23	0.27
Same and Different	0.88	0.53	0.17	0.17
Top and Bottom 2D	0.95	0.60	0.57	0.23
Top and Bottom 3D	0.93	0.50	0.03	0.20

Figure 4: Concepts utilisés dans ConceptARC et résultats de certaines approches
Ici les résultats correspondent aux pourcentages des tâches résolues pour chaque classe. Pour plus de détails, voir [4]

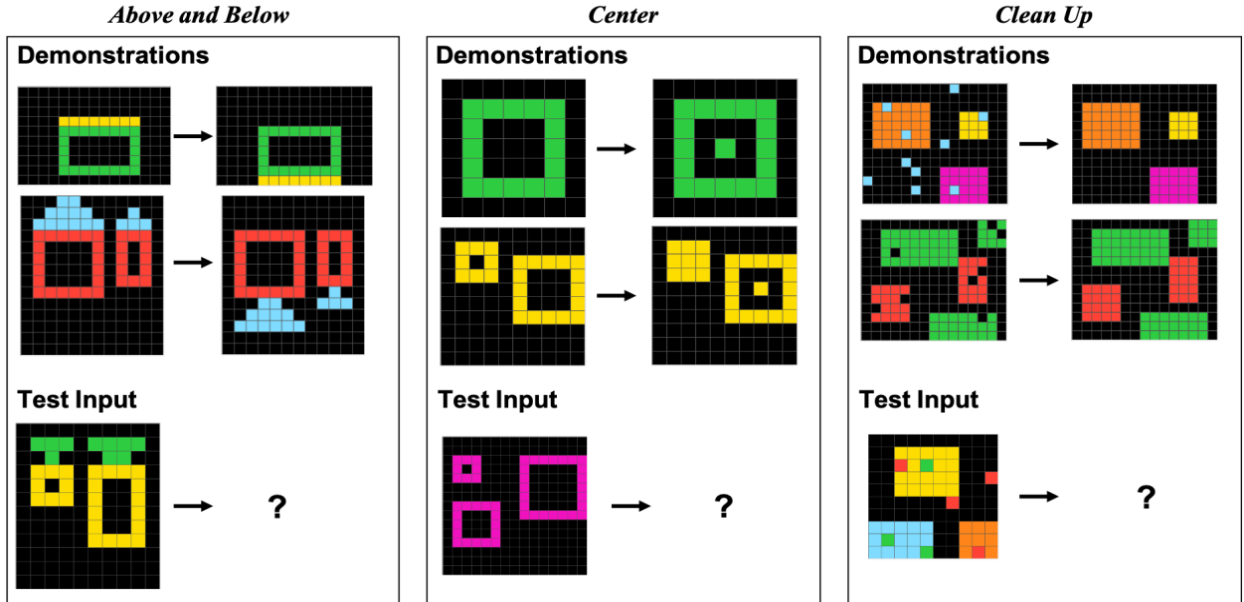


Figure 5: Exemples du dataset ConceptARC

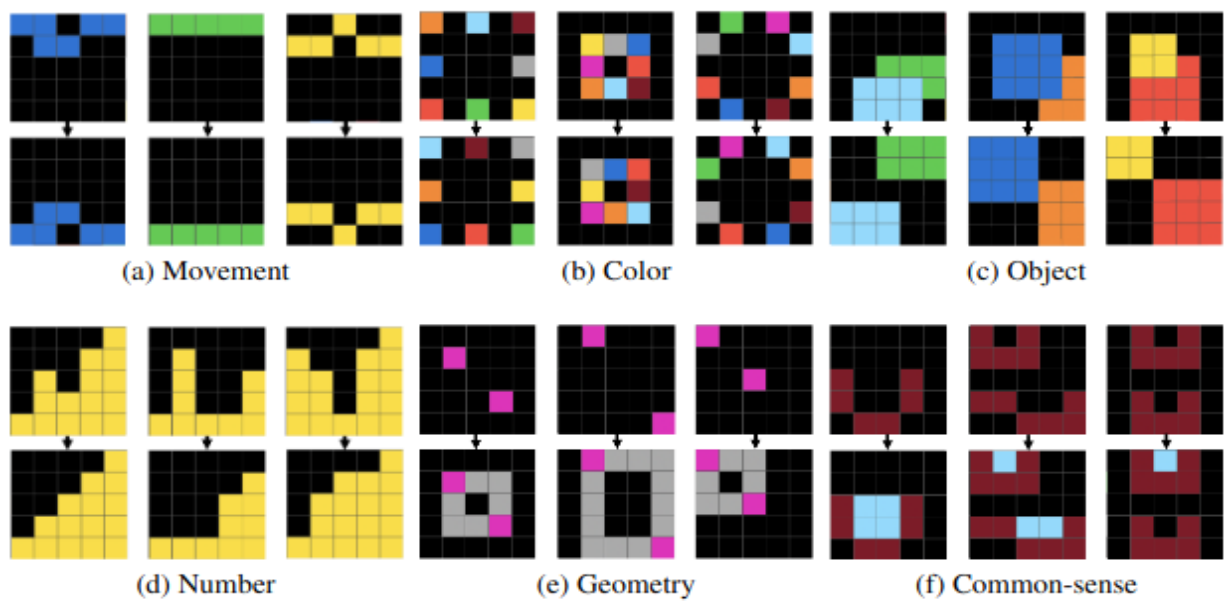


Figure 6: Representative Mini-ARC examples for each category

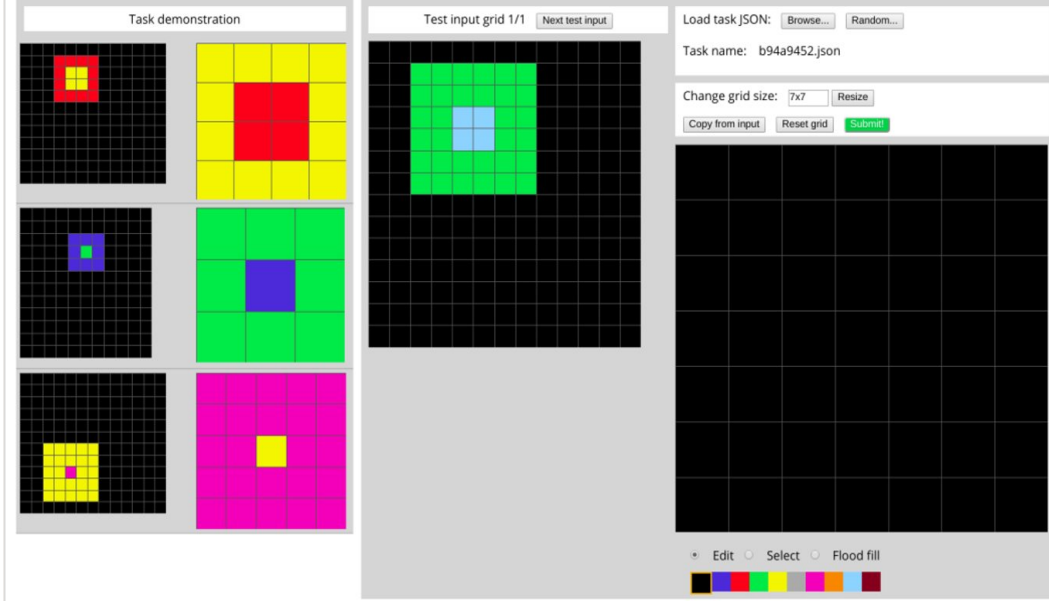


Figure 7: Tâche b94a9452.json du dataset d'entraînement

```

Pair    ::= InOut(in : Grid, out : Grid)
Grid    ::= Grid(size : Vector, color : C, layers : list(Object))
Object  ::= PosShape(pos : Vector, shape : Shape)
Shape   ::= Point(color : C)
          | Rectangle(size : Vector, color : C, mask : Mask)
Vector  ::= Vec(i : N, j : N)
Mask    ::= Bitmap(bitmap : M)
          | Full | Border
          | EvenCheckboard | OddCheckboard
          | PlusCross | TimesCross

```

Figure 8: Type de données utilisés dans les modèles de grilles dans l'approche de Sébastien Ferré [7]

```

InOut(
  Grid(Vec(12, ?), black,
    [ PosShape(Vec(? , ?), Rectangle(Vec(? , ?), ? , Full)),
      PosShape(Vec(? , ?), Rectangle(Vec(? , ?), ? , Full) ])),
  Grid(layers[1].shape.size, layers[0].shape.color,
    [ PosShape(
      Vec(layers[0].pos.i - layers[1].pos.i, layers[0].pos.j - layers[1].pos.j),
      Rectangle(layers[0].size, layers[1].color, Full) ) ])).

```

Figure 9: Model description of task
b94a9452 de l'approche de Sébastien Ferré [7]

- $L_{\mathbb{N}}(n) = 2 \log(n + 1) + 1$: universal encoding of natural intergers (including zero), more precisely the gamma encoding shifted by one to include zero;
- $L_P(x \in X) = -\log P(x)$: encoding based on a probability distribution P over elements X ;
- $L_X(x \in X) = \log |X|$: encoding based on a uniform distribution over elements X .

Figure 10: Règles pour le calcul de la longueur de description pour des types élémentaires de l'approche de Sébastien Ferré

step	$\hat{L}(M, D)$	refinement
0	2.000	
1	1.272	$in.layer[1] = \mathbf{PosShape}(?, \mathbf{Rectangle}(?, ?, ?))$
2	1.031	$out.size = layer[1].shape.size$
3	0.814	$out.layer[0] = \mathbf{PosShape}(?, \mathbf{Rectangle}(?, ?, ?))$
4	0.732	$out.layer[0].shape.color = in.layer[1].shape.color$
5	0.688	$in.layer[0] = \mathbf{PosShape}(?, \mathbf{Rectangle}(?, ?, ?))$
6	0.549	$out.color = in.layer[0].shape.color$
7	0.423	$out.layer[0].shape.size = in.layer[0].shape.size$
8	0.381	$out.layer[0].shape.mask = in.layer[0].shape.mask$
9	0.361	$out.layer[0].shape.pos = \mathbf{Vec}(?, ?)$
10	0.310	$out.layer[0].shape.pos.i = in.layer[0].shape.pos.i - in.layer[1].shape.pos.i$
11	0.259	$out.layer[0].shape.pos.j = in.layer[0].shape.pos.j - in.layer[1].shape.pos.j$
12	0.252	$in.layer[0].shape.mask = \mathbf{Full}$
13	0.246	$in.layer[1].shape.mask = \mathbf{Full}$
14	0.241	$in.color = black$
15	0.238	$in.size = \mathbf{Vec}(?, ?)$
16	0.212	$in.size.i = 12$
17	0.209	$in.layer[0].shape.pos = \mathbf{Vec}(?, ?)$
18	0.205	$in.layer[0].shape.size = \mathbf{Vec}(?, ?)$
19	0.202	$in.layer[1].shape.pos = \mathbf{Vec}(?, ?)$
20	0.199	$in.layer[1].shape.size = \mathbf{Vec}(?, ?)$

Figure 11: Itérations pour résoudre la tâche b94a9452.json du dataset d'entraînement dans l'approche de Sébastien Ferré [7]

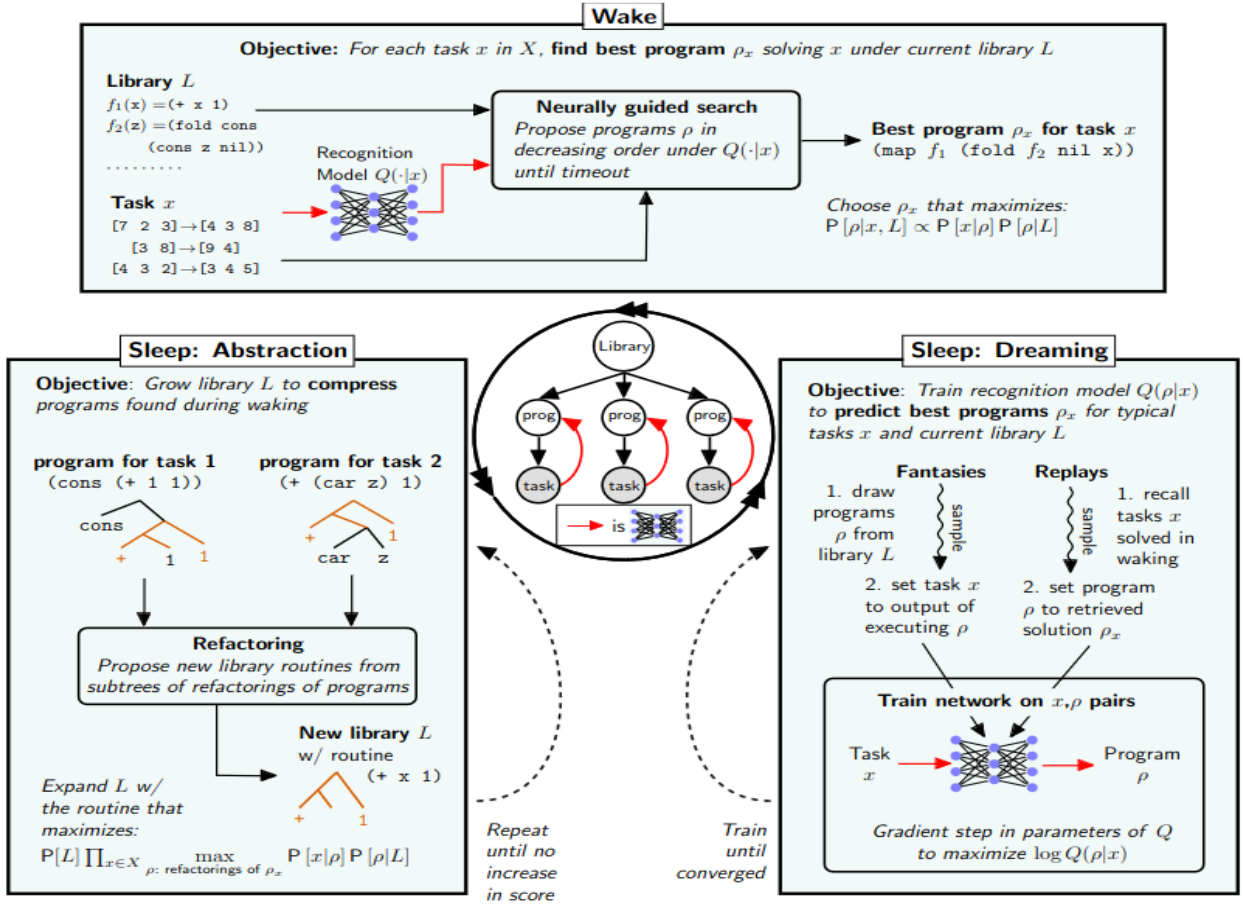


Figure 12: Architecture de DreamCoder [8]

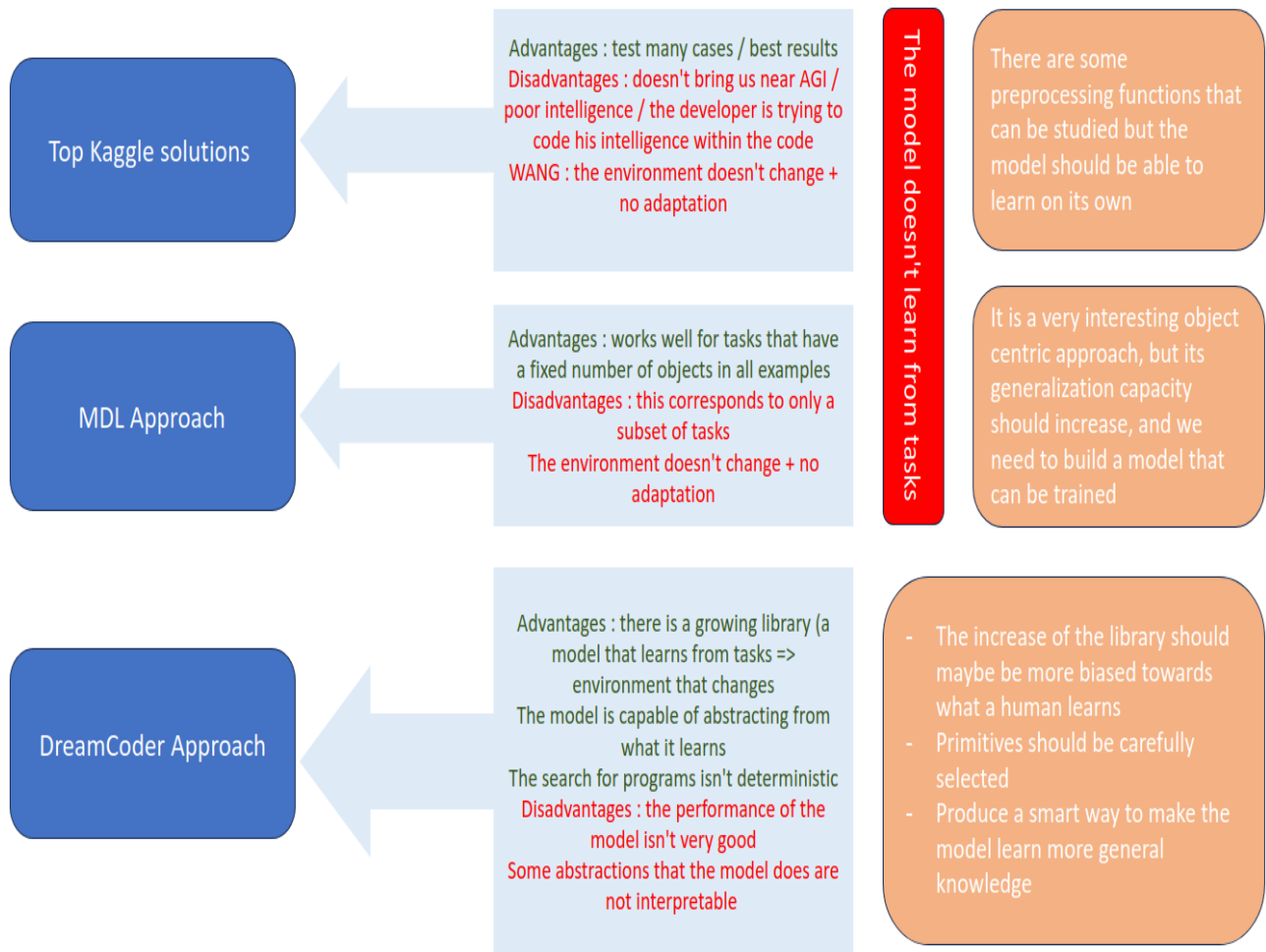
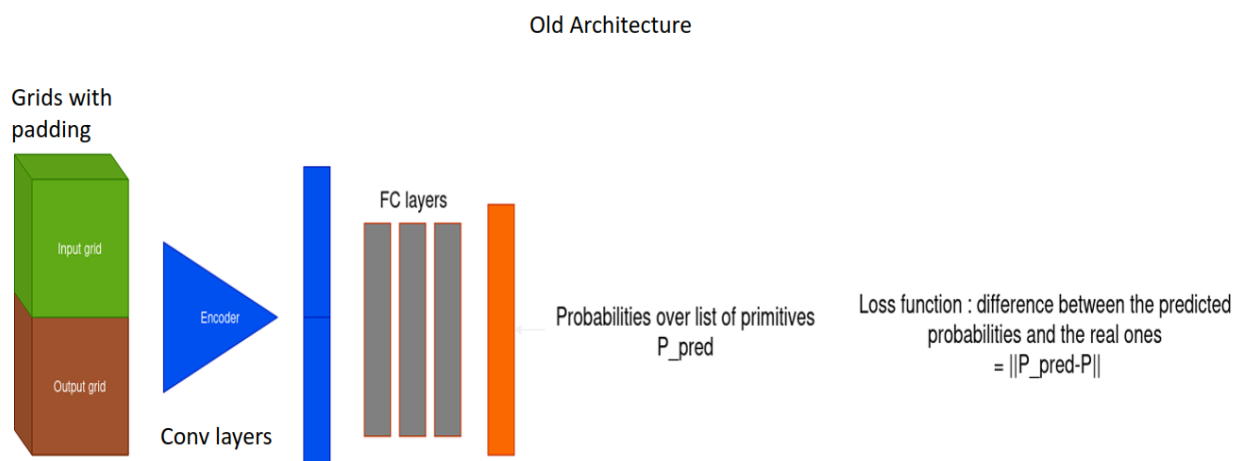


Figure 13: Comparaison personnelle des approches



For each example, we obtain an encoding vector, then we calculate the mean over all examples

Figure 14: Architecture originale du réseau de neurones dans DreamCoder

total_params 123520		
Layer (type)	Output Shape	Param #
Conv2d-1	[-1, 64, 30, 30]	12,736
ReLU-2	[-1, 64, 30, 30]	0
MaxPool2d-3	[-1, 64, 15, 15]	0
Conv2d-4	[-1, 64, 15, 15]	36,928
ReLU-5	[-1, 64, 15, 15]	0
MaxPool2d-6	[-1, 64, 7, 7]	0
Conv2d-7	[-1, 64, 7, 7]	36,928
ReLU-8	[-1, 64, 7, 7]	0
MaxPool2d-9	[-1, 64, 3, 3]	0
Conv2d-10	[-1, 64, 3, 3]	36,928
ReLU-11	[-1, 64, 3, 3]	0
MaxPool2d-12	[-1, 64, 1, 1]	0
Flatten-13	[-1, 64]	0
Total params: 123,520		
Trainable params: 123,520		
Non-trainable params: 0		
Input size (MB): 0.08		
Forward/backward pass size (MB): 1.29		
Params size (MB): 0.47		
Estimated Total Size (MB): 1.84		

Figure 15: Composition de l'encodeur dans l'architecture originale de Dreamcoder

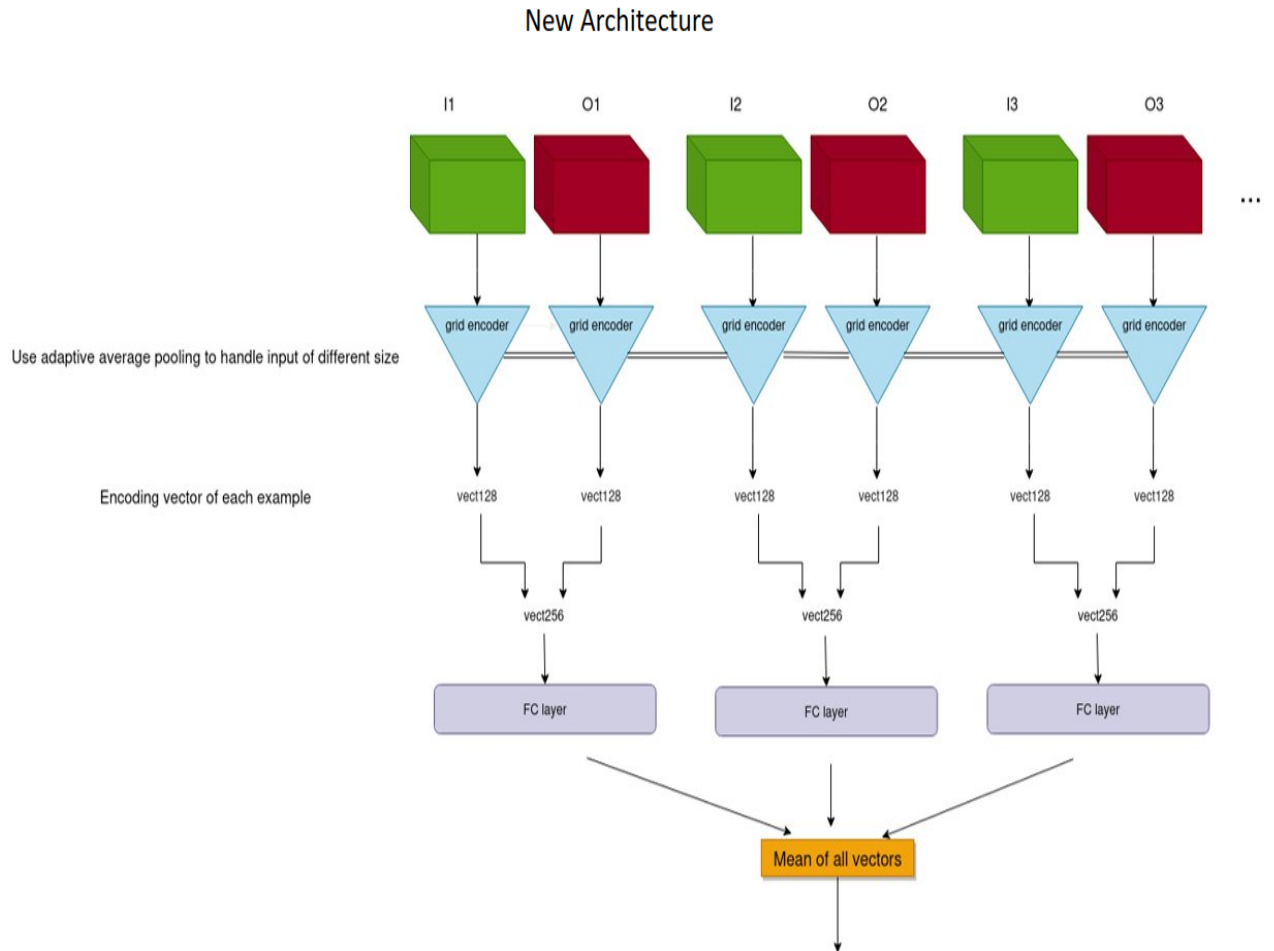


Figure 16: Nouvelle architecture proposée pour DreamCoder

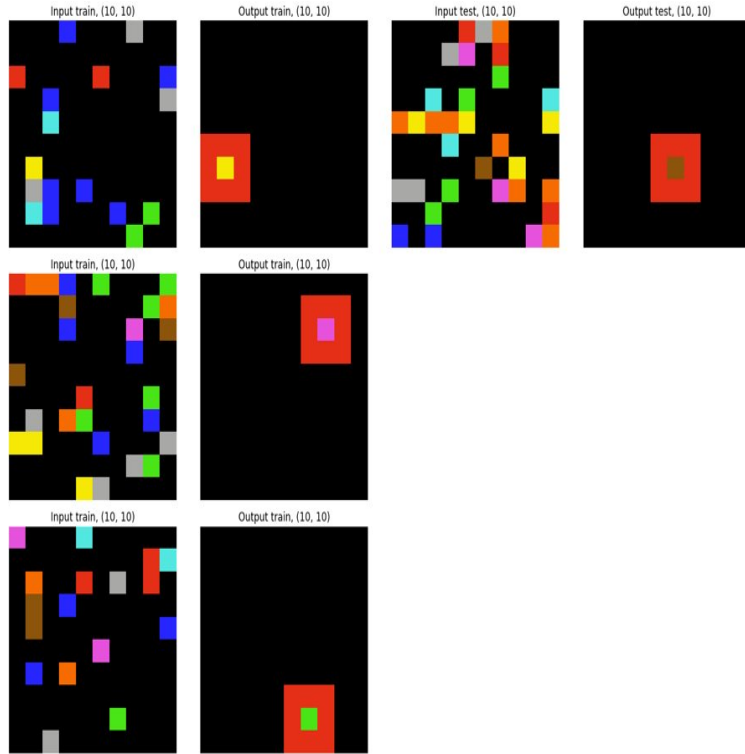


Figure 17: Tâche ARC du dataset d'entraînement

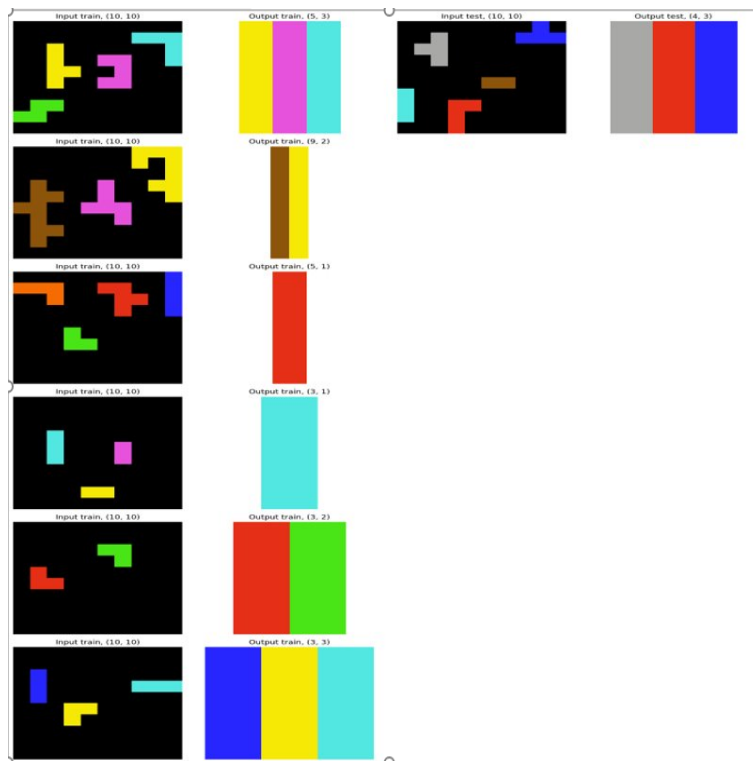


Figure 18: Tâche ARC du dataset d'entraînement

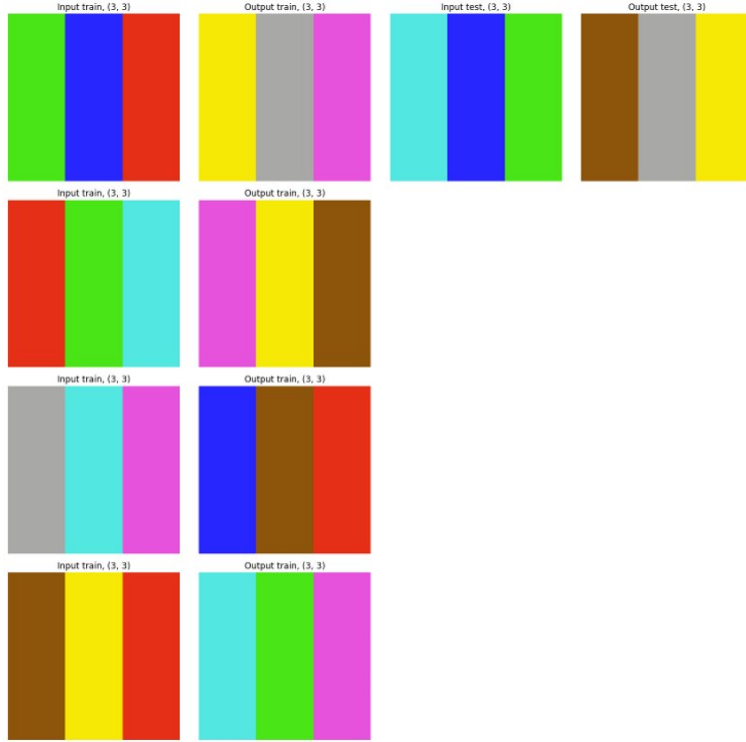


Figure 19: tâche ARC du dataset d'entraînement

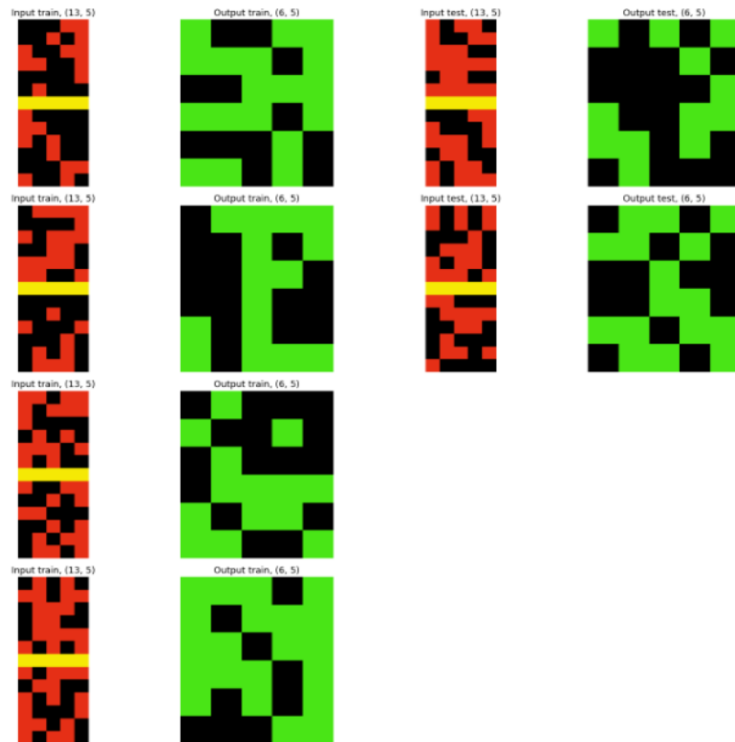


Figure 20: Tâche ARC du dataset d'entraînement

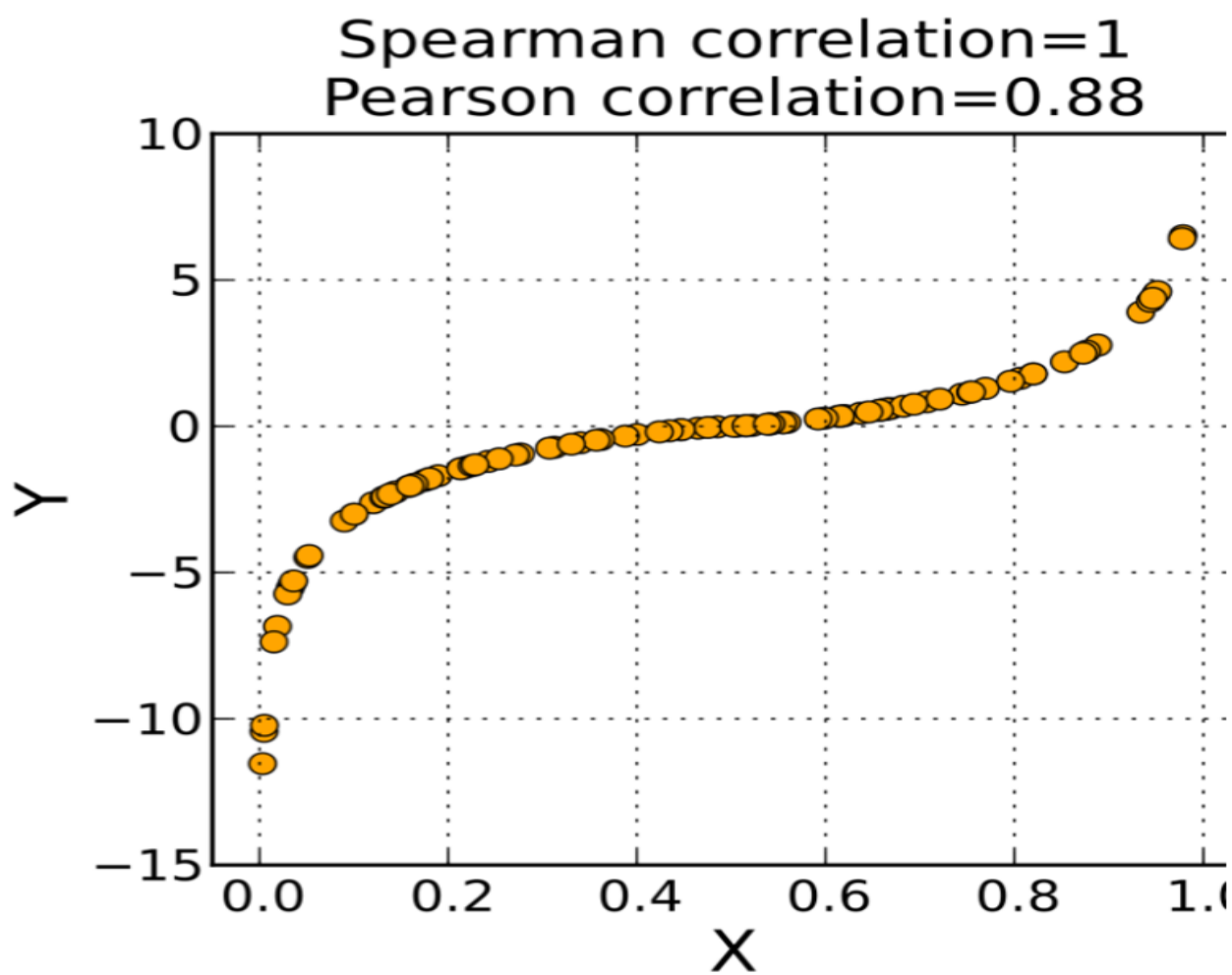


Figure 21: Graphe indiquant le coefficient de corrélation de Spearman et de Pearson entre deux variables statistiques X et Y

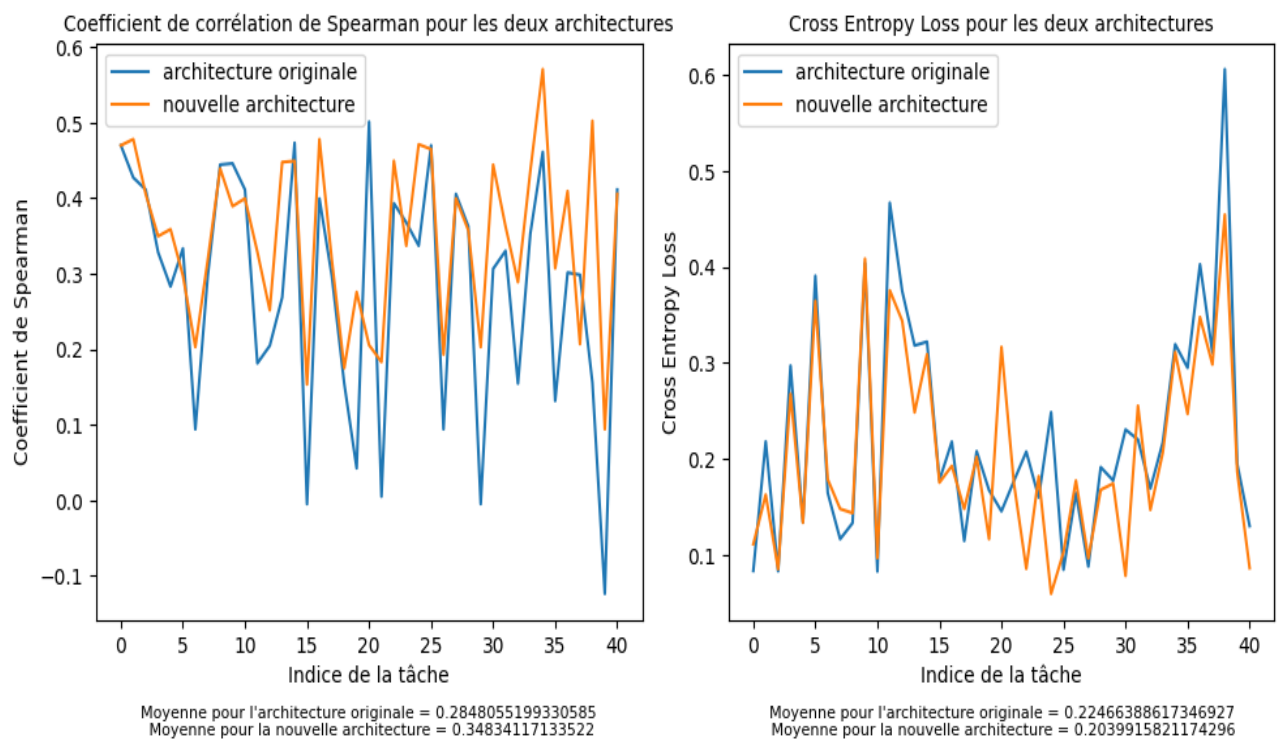


Figure 22: Evaluation du coefficient de Spearman et de Cross Entropy Loss pour les deux architectures

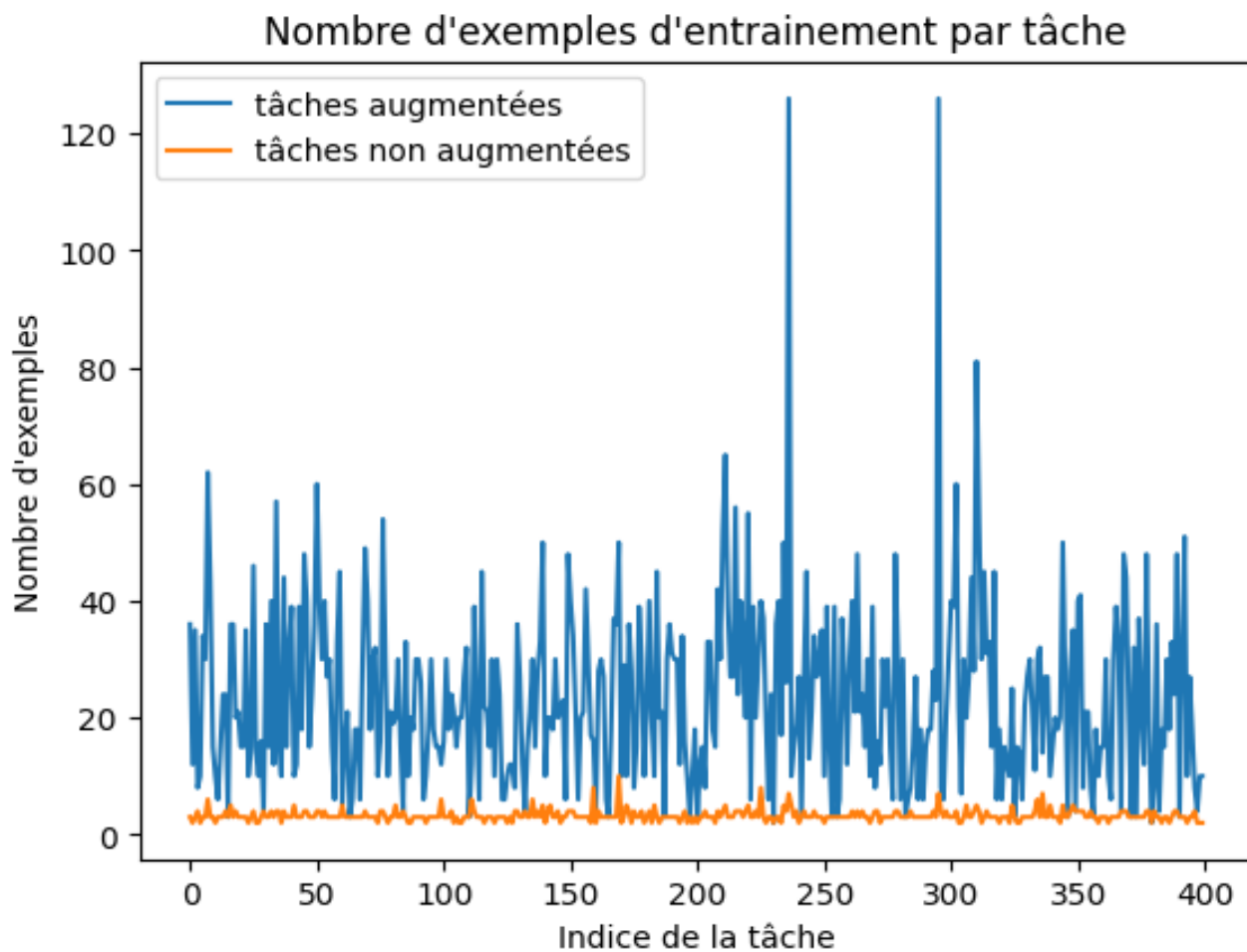


Figure 23: Nombre d'exemples d'entrainement par tâche

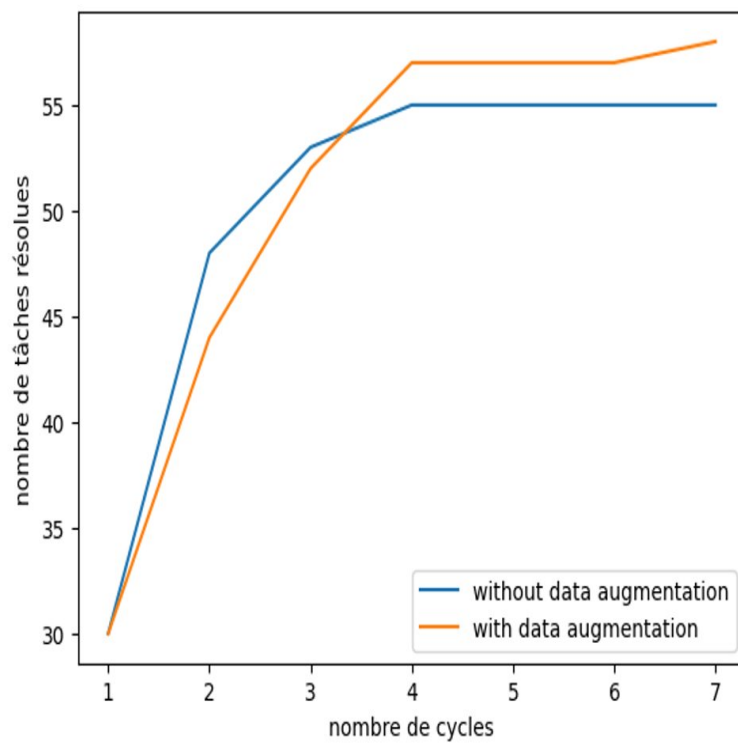


Figure 24: Comparaison entre les tâches résolues avec et sans augmentation des données

List Processing

Sum List

[1 2 3] → 6

[4 6 8 1] → 17

Double

[1 2 3] → [2 4 6]

[4 5 1] → [8 10 2]

Check Evens

[0 2 3] → [T T F]

[2 9 6] → [T F T]

Figure 25: Exemples du domaine "list processing"

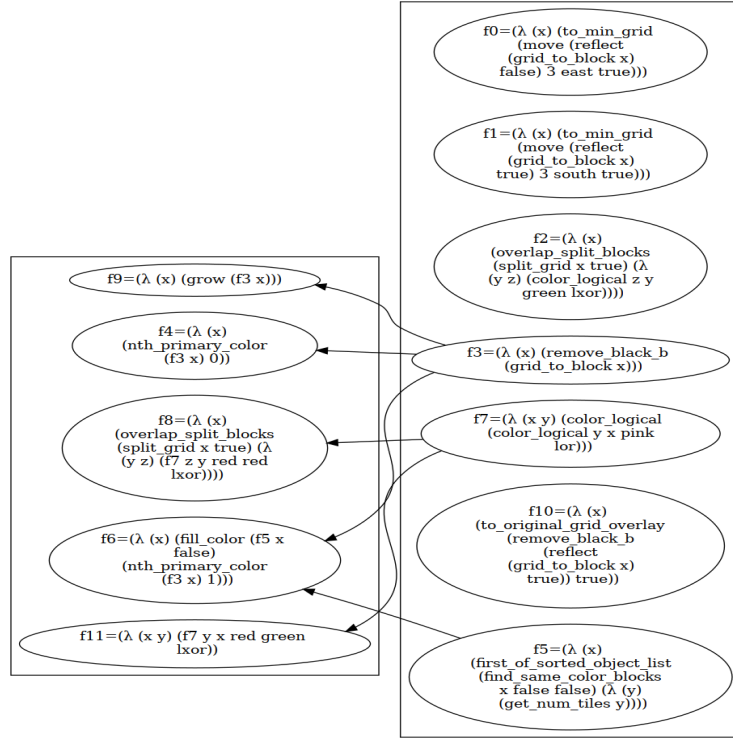


Figure 26: Quelques concepts ajoutés à la librairie après 6 cycles

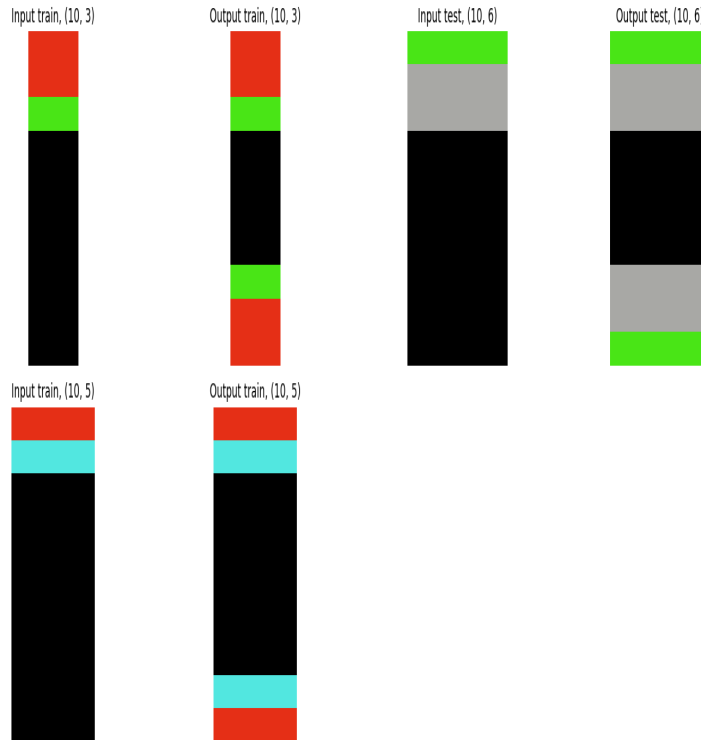


Figure 27: tâche 496994bd.json du dataset d'entraînement

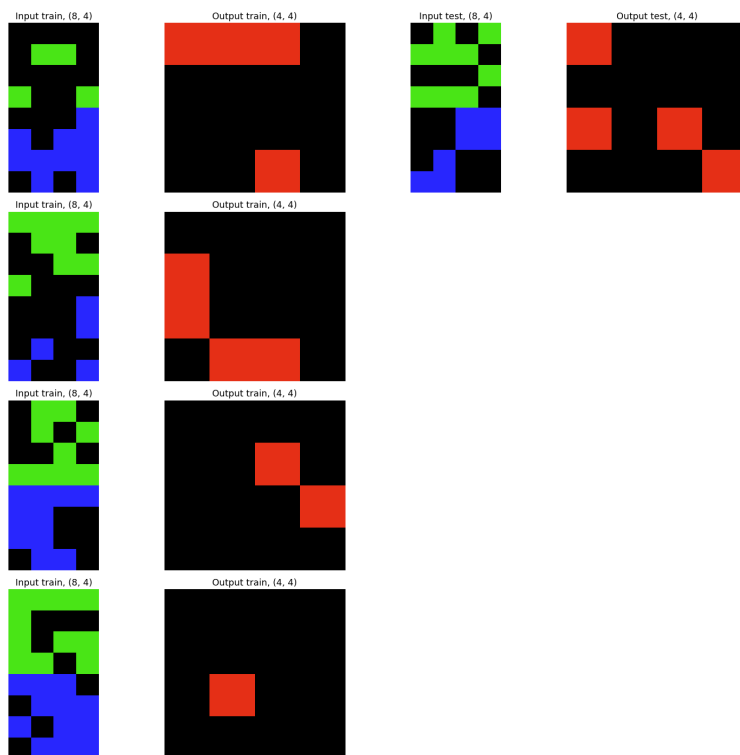


Figure 28: tâche 94f9d214.json du dataset d'entraînement